Tensorflow II

Vision Transformers (ViT)

Fatemeh Dehrouyeh



Contents

1. Vision Transformers
   - Motivation
   - Image Patches
   - Patch Embeddings
   - Self-Attention
   - Transformer Encoder Block
   - Building a Small Vision Transformer
   - The Classification Token
   - CNNs vs. Vision Transformers
   - Pretrained Vision Transformers
   - Using KerasHub's pretrained ViT

Current Section

1. Vision Transformers
   - Motivation
   - Image Patches
   - Patch Embeddings
   - Self-Attention
   - Transformer Encoder Block
   - Building a Small Vision Transformer
   - The Classification Token
   - CNNs vs. Vision Transformers
   - Pretrained Vision Transformers
   - Using KerasHub's pretrained ViT

Current Subsection

1. Vision Transformers
   - Motivation
   - Image Patches
   - Patch Embeddings
   - Self-Attention
   - Transformer Encoder Block
   - Building a Small Vision Transformer
   - The Classification Token
   - CNNs vs. Vision Transformers
   - Pretrained Vision Transformers
   - Using KerasHub's pretrained ViT

From CNNs to Vision Transformers

CNNs process images using local convolution filters.

Vision Transformers process images differently:

$$\text{Image} \rightarrow \text{Patches} \rightarrow \text{Tokens} \rightarrow \text{Transformer} \rightarrow \text{Prediction}$$

Core Idea

A Vision Transformer treats an image as a sequence of patches, similar to how a language transformer treats text as a sequence of words.

Why Vision Transformers?

CNNs are excellent at learning local patterns.

However, transformers are good at modeling relationships between all parts of an input sequence.

For images, this means a model can compare distant image regions directly.

Key Difference

CNNs use local filters, while Vision Transformers use attention between image patches.

Main Idea of a Vision Transformer

A Vision Transformer converts an image into small patches.

Each patch becomes a vector called a **token**.

The transformer then learns relationships among these tokens using **self-attention**.

$$\boxed{\text{Image} \rightarrow \text{Patch Tokens} \rightarrow \text{Self-Attention} \rightarrow \text{Classification}}$$

Current Subsection

1. Vision Transformers
   - Motivation
   - Image Patches
   - Patch Embeddings
   - Self-Attention
   - Transformer Encoder Block
   - Building a Small Vision Transformer
   - The Classification Token
   - CNNs vs. Vision Transformers
   - Pretrained Vision Transformers
   - Using KerasHub's pretrained ViT

Splitting an Image into Patches

Instead of processing pixels one by one, a Vision Transformer splits the image into fixed-size patches.

For example, a 224×224 image with 16×16 patches gives:

$$\frac{224}{16} \times \frac{224}{16} = 14 \times 14 = 196$$

patches.

Interpretation

Each patch is treated like one token in a sequence.

Sample Image

```python
import tensorflow as tf
import matplotlib.pyplot as plt
import urllib.request
from PIL import Image
import numpy as np

# Image hosted on GitHub (PyTorch Hub repo)
url = "https://github.com/pytorch/hub/raw/master/images/dog.jpg"

# Download the image
urllib.request.urlretrieve(url, "dog.jpg")

# Load image
img = Image.open("dog.jpg").convert("RGB")
img = np.array(img)

# Convert to float32 in [0, 1]
img_float = tf.image.convert_image_dtype(img, tf.float32)

# Display image
plt.imshow(img_float)
plt.axis("off")
plt.title("Dog - PyTorch GitHub Sample")
plt.show()
```

Sample Image—Result



Code

```
import tensorflow as tf

image = tf.image.resize(img_float, (224, 224))

patch_size = 16

patches = tf.image.extract_patches(
    images=tf.expand_dims(image, axis=0),
    sizes=[1, patch_size, patch_size, 1],
    strides=[1, patch_size, patch_size, 1],
    rates=[1, 1, 1, 1],
    padding="VALID"
)

print("Patches shape:", patches.shape)
```

```
Patches shape: (1, 14, 14, 768)
```

Vision Transformer: Image Patch Extraction

- `patch_size = 16`: Defines each patch as 16×16 pixels.
- `tf.image.extract_patches()`: Extracts small patches from the image.
- `tf.expand_dims(image, axis=0)`: Adds a batch dimension to the image.
- `sizes=[1,16,16,1]`: Defines the extraction window size.
- `strides=[1,16,16,1]`: Moves 16 pixels at a time, preventing overlapping patches.
- `rates=[1,1,1,1]`: Includes every pixel without skipping.
- `padding="VALID"`: Extracts only complete patches without padding.
- `patches.shape = (1,14,14,768)`: Represents 1 image, 196 patches, and 768 RGB values per patch.

What Is Inside One Patch?

A patch contains a small block of pixels.

For an RGB image and a 16×16 patch:

$$16 \times 16 \times 3 = 768$$

numbers.

So each patch can be flattened into a vector of length 768.

Key Idea

A patch token is a vector representation of a small image region.

Code

```
num_patches_h = 224 // patch_size
num_patches_w = 224 // patch_size
num_patches = num_patches_h * num_patches_w
patch_dim = patch_size * patch_size * 3

patches_reshaped = tf.reshape(
    patches,
    (1, num_patches, patch_dim)
)

print("Number of patches:", num_patches)
print("Patch vector length:", patch_dim)
print("Patch sequence shape:", patches_reshaped.shape)
```

```
Number of patches: 196
Patch vector length: 768
Patch sequence shape: (1, 196, 768)
```

Visualizing Image Patches

Patch extraction turns one image into many small image pieces.

For a 224×224 image with 16×16 patches:

$$196$$

patches are created.

Transformer View

The image is now a sequence of 196 patch tokens.

Code

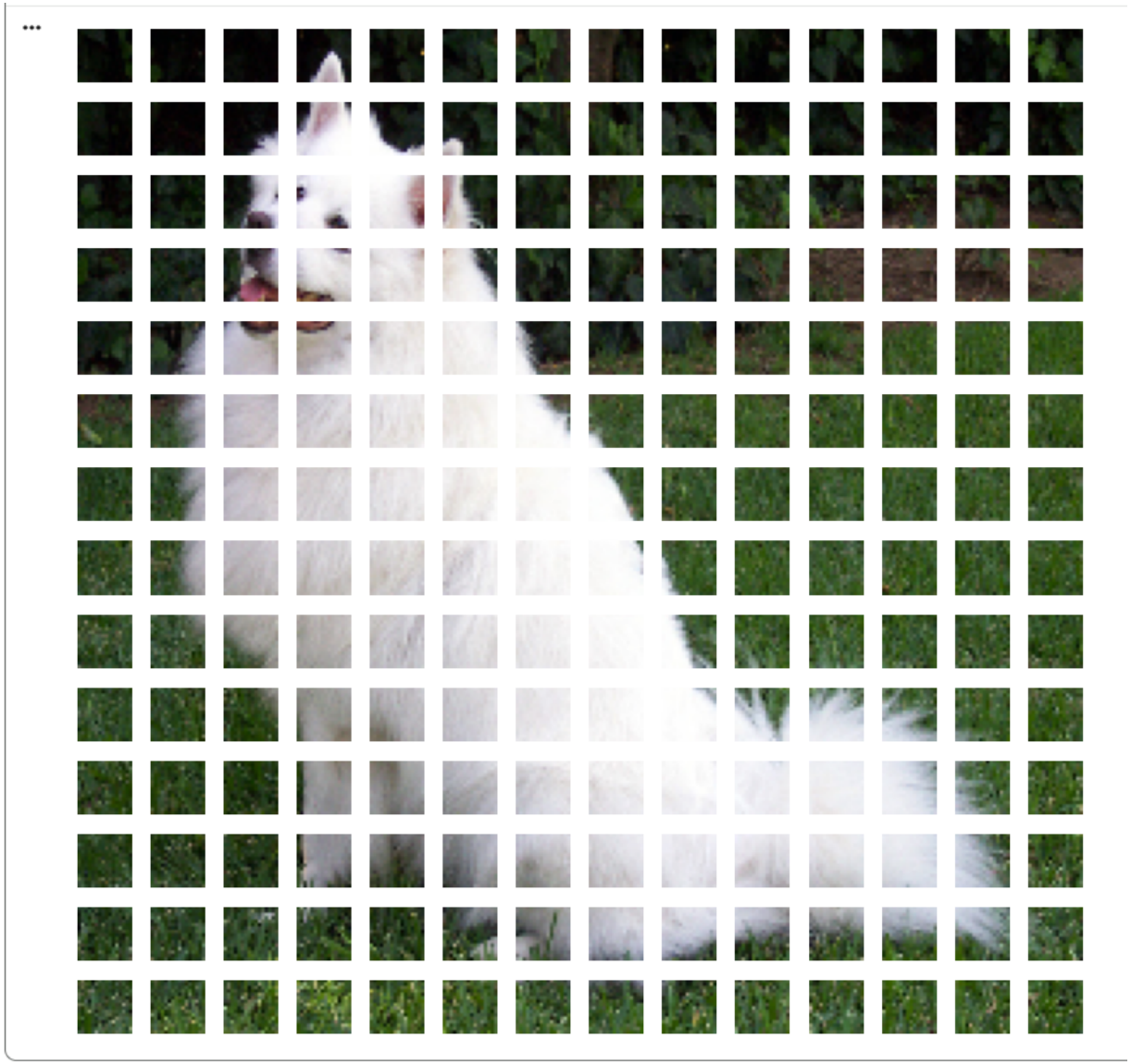
```
patches_for_display = tf.reshape(
    patches,
    (num_patches_h, num_patches_w, patch_size, patch_size, 3)
)

plt.figure(figsize=(8, 8))

for i in range(num_patches):
    plt.subplot(num_patches_h, num_patches_w, i + 1)
    row = i // num_patches_w
    col = i % num_patches_w
    plt.imshow(patches_for_display[row, col])
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Code—Result



Current Subsection

1. Vision Transformers
   - Motivation
   - Image Patches
   - Patch Embeddings
   - Self-Attention
   - Transformer Encoder Block
   - Building a Small Vision Transformer
   - The Classification Token
   - CNNs vs. Vision Transformers
   - Pretrained Vision Transformers
   - Using KerasHub's pretrained ViT

From Patches to Embeddings

Raw patch vectors can be large.

For example:

$$16 \times 16 \times 3 = 768$$

numbers per patch.

Vision Transformers usually project each patch vector into an embedding space:

$$\text{Patch Vector} \rightarrow \text{Dense Projection} \rightarrow \text{Patch Embedding}$$

Key Idea

Patch embeddings are the input tokens given to the transformer.

Code

```
projection_dim = 64

projection = tf.keras.layers.Dense(projection_dim)

patch_embeddings = projection(patches_reshaped)

print("Patch embeddings shape:", patch_embeddings.shape)
```

```
Patch embeddings shape: (1, 196, 64)
```

The Dense layer's learnable parameters (weights and biases) are shared across all patches, images, and batches, and are updated during training.

Patch Projection: From 768 to 64

We choose the embedding dimension:

`projection_dim = 64`

Then we create a Dense layer:

`Dense(64)`

Each raw patch contains:

$$16 \times 16 \times 3 = 768$$

values.
So before projection, the patch sequence has shape:

$$\boxed{(1, 196, 768)}$$

The Dense layer transforms each patch independently:

$$\boxed{768 \rightarrow 64}$$

for all 196 patches.
Therefore, the output shape becomes:

$$\boxed{(1, 196, 64)}$$

Each of the 196 image patches is now represented by a learned 64-dimensional embedding.

Learnable Weights in Patch Projection

The patch projection is a trainable Dense layer:

```
projection = Dense(64)
```

Each patch starts as a vector of length: 768

The Dense layer applies:

$$\boxed{y = xW + b}$$

where:

- x = patch vector of length 768
- W = learnable weight matrix
- b = learnable bias
- y = embedding vector of length 64

The weight matrix has shape: $\boxed{768 \times 64}$

and the bias has shape: $\boxed{64}$

Total trainable parameters: $768 \times 64 + 64 = \boxed{49,216}$

Important: The same learned projection is applied to all 196 patches.

Positional Embeddings

A transformer receives a sequence of patch tokens.

However, the model also needs to know where each patch came from.

Positional embeddings add location information to patch embeddings.

$$\text{Patch Embedding} + \text{Position Embedding} = \text{Transformer Input}$$

Why?

Without position information, the transformer does not naturally know the spatial order of image patches.

Code

```
position_embedding = tf.keras.layers.Embedding(
    input_dim=num_patches,
    output_dim=projection_dim
)

positions = tf.range(start=0, limit=num_patches, delta=1)

encoded_patches = patch_embeddings + position_embedding(positions)

print("Encoded patches shape:", encoded_patches.shape)
```

```
Encoded patches shape: (1, 196, 64)
```

Learned Positional Encoding

The transformer needs to know where each image patch came from.

We create position IDs:

$$0, 1, 2, \ldots, 195$$

using:

```
tf.range(start=0, limit=num_patches, delta=1)
```

A trainable embedding layer is then created:

```
Embedding(196, 64)
```

This means:

$$196 \text{ positions} \times 64 \text{ values per position}$$

Each position gets its own learned 64-dimensional vector.

$$\boxed{\text{Encoded Patch} = \text{Patch Embedding} + \text{Learned Position Embedding}}$$

Important

This is learned positional encoding.
It is **not** sinusoidal sine/cosine positional encoding.

Current Subsection

1. Vision Transformers
   - Motivation
   - Image Patches
   - Patch Embeddings
   - Self-Attention
   - Transformer Encoder Block
   - Building a Small Vision Transformer
   - The Classification Token
   - CNNs vs. Vision Transformers
   - Pretrained Vision Transformers
   - Using KerasHub's pretrained ViT

What Is Self-Attention?

Self-attention allows each patch token to compare itself with every other patch token.

This helps the model learn relationships between distant parts of the image.

$$\text{Patch}_i \quad \leftrightarrow \quad \text{Patch}_j$$

Key Idea

Attention tells the model which patches are important for understanding other patches.

Code

```
attention_layer = tf.keras.layers.MultiHeadAttention(
    num_heads=4,
    key_dim=projection_dim
)

attention_output = attention_layer(
    encoded_patches,
    encoded_patches
)

print("Attention output shape:", attention_output.shape)
```

```
Attention output shape: (1, 196, 64)
```

This is self-attention because the same encoded_patches tensor is used as both the query and the value, and Keras also uses it for the key when no separate key is provided.

Multi-Head Attention

Multi-head attention uses several attention mechanisms in parallel.

Each head can focus on different relationships between patches.

- one head may focus on object parts,
- another may focus on background structure,
- another may focus on long-range relationships.

Intuition

Multiple heads allow the model to look at the image in different ways.

Each head learns its own projection:

$$\boxed{64 \rightarrow 64}$$

For example:

$$Q_1 = XW_{Q_1}, \qquad Q_2 = XW_{Q_2}, \qquad Q_3 = XW_{Q_3}, \qquad Q_4 = XW_{Q_4}$$

Each W_Q is different.
The four heads allow the model to learn different relationships between the 196 image patches.

Multi-Head Self-Attention Output

- **Attention Scores:** QK^T measures the relationship between image patches.
- **Attention Weights:** $A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$ determines how much attention each patch gives to others.
- **Value Multiplication:** $Z = AV$ produces weighted sums of Value vectors.
- **Multiple Heads:** Each of the 4 heads independently calculates $Z_i = A_i V_i$.
- **Final Projection:** $\text{Concat}(Z_1, Z_2, Z_3, Z_4)W_O + b_O$ combines the heads into updated embeddings.
- **Output Shape:** $(1, 196, 64)$ represents 1 image, 196 patches, and 64 updated features per patch.

Current Subsection

1. Vision Transformers
   - Motivation
   - Image Patches
   - Patch Embeddings
   - Self-Attention
   - Transformer Encoder Block
   - Building a Small Vision Transformer
   - The Classification Token
   - CNNs vs. Vision Transformers
   - Pretrained Vision Transformers
   - Using KerasHub's pretrained ViT

Transformer Encoder Block

A Vision Transformer is built from repeated transformer encoder blocks.

Each block usually contains:

- layer normalization,
- multi-head self-attention,
- residual connections,
- feed-forward neural network.

$$\text{Tokens} \rightarrow \text{Attention} \rightarrow \text{MLP} \rightarrow \text{Updated Tokens}$$

Code

```python
def transformer_encoder(inputs, num_heads, projection_dim, mlp_dim):
    # Self-attention block
    x1 = tf.keras.layers.LayerNormalization()(inputs)
    attention_output = tf.keras.layers.MultiHeadAttention(
        num_heads=num_heads,
        key_dim=projection_dim
    )(x1, x1)

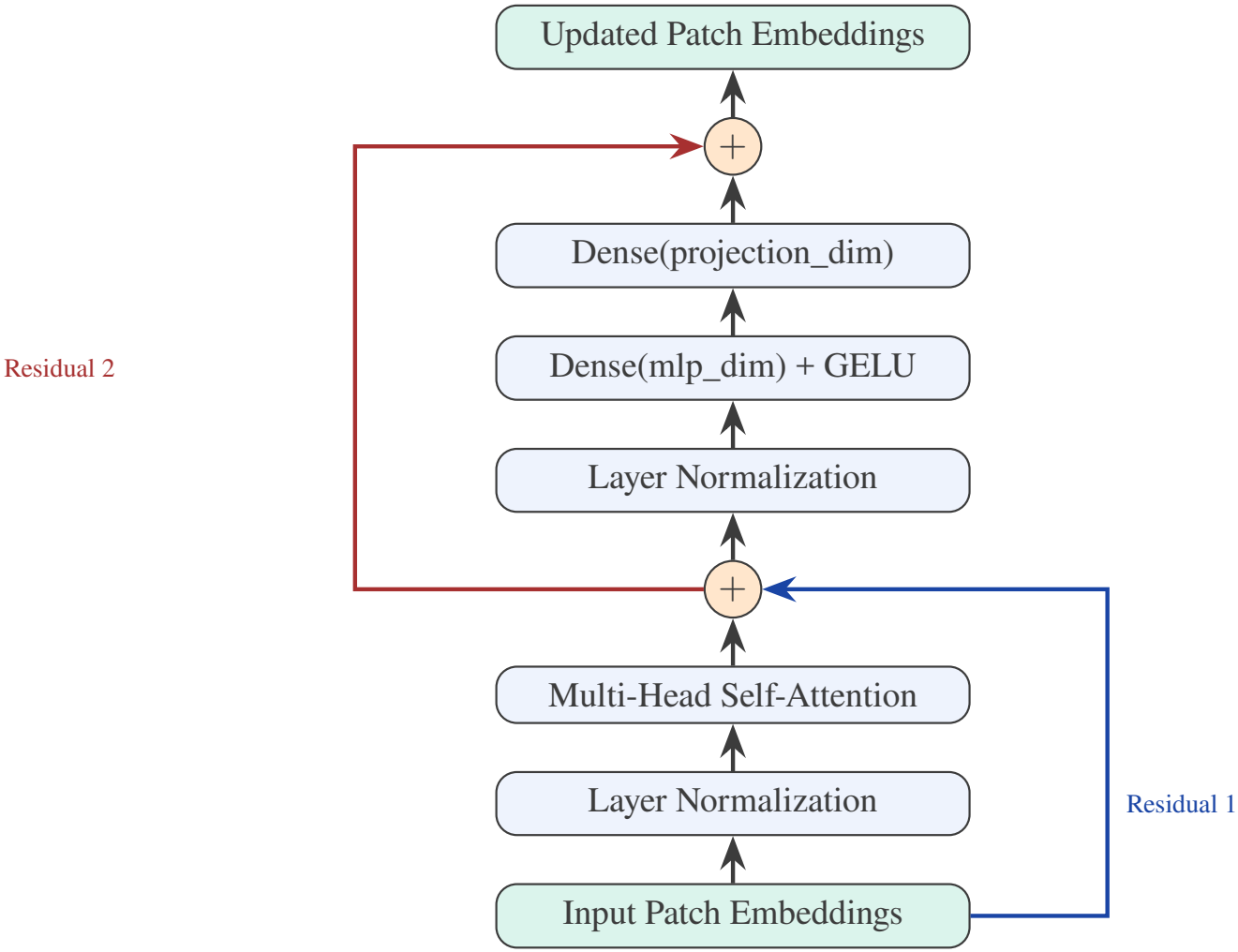
    # Residual connection
    x2 = tf.keras.layers.Add()([attention_output, inputs])

    # MLP block
    x3 = tf.keras.layers.LayerNormalization()(x2)
    x3 = tf.keras.layers.Dense(mlp_dim, activation="gelu")(x3)
    x3 = tf.keras.layers.Dense(projection_dim)(x3)

    # Residual connection
    outputs = tf.keras.layers.Add()([x3, x2])

    return outputs
```

Transformer Encoder Block



Residual Connections

A residual connection adds the input of a block back to its output.

$$\text{Output} = F(x) + x$$

This helps information and gradients flow through deep networks.

In Transformers

Residual connections are used around both the attention block and the MLP block.

Code

```
x = encoded_patches

encoder_output = transformer_encoder(
    inputs=x,
    num_heads=4,
    projection_dim=64,
    mlp_dim=128
)

print("Input shape:", x.shape)
print("Encoder output shape:", encoder_output.shape)
```

```
Input shape: (1, 196, 64)
Encoder output shape: (1, 196, 64)
```

MLP Block Inside a Transformer

After self-attention, each token passes through a small feed-forward network.

This is often called the **MLP block**.

$$\text{Token} \rightarrow \text{Dense} \rightarrow \text{GELU} \rightarrow \text{Dense} \rightarrow \text{Updated Token}$$

Key Idea

Attention mixes information between tokens, while the MLP transforms each token representation.

Code

```
mlp = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation="gelu"),
    tf.keras.layers.Dense(64)
])

mlp_output = mlp(encoded_patches)

print(mlp_output.shape)
```

```
(1, 196, 64)
```

Current Subsection

1. Vision Transformers
   - Motivation
   - Image Patches
   - Patch Embeddings
   - Self-Attention
   - Transformer Encoder Block
   - Building a Small Vision Transformer
   - The Classification Token
   - CNNs vs. Vision Transformers
   - Pretrained Vision Transformers
   - Using KerasHub's pretrained ViT

Vision Transformer Pipeline

A simple Vision Transformer follows this pipeline:

$$\text{Image} \rightarrow \text{Patches} \rightarrow \text{Patch Embeddings} \rightarrow \text{Position Embeddings} \rightarrow \text{Transformer Blocks} \rightarrow \text{Classifier}$$

Goal

Convert an image into a sequence, process the sequence with attention, and classify the image.

Patch Extraction Layer

To build a Vision Transformer in TensorFlow, we can create a custom layer that extracts patches from an input image.

This layer converts an image from

$$(H, W, C)$$

into a sequence of flattened patches:

$$(N_{\text{patches}},\ \text{patch dimension})$$

Key Idea

The patch extraction layer prepares the image for transformer-style sequence processing.

Code

```python
class Patches(tf.keras.layers.Layer):
    def __init__(self, patch_size):
        super().__init__()
        self.patch_size = patch_size

    def call(self, images):
        batch_size = tf.shape(images)[0]

        patches = tf.image.extract_patches(
            images=images,
            sizes=[1, self.patch_size, self.patch_size, 1],
            strides=[1, self.patch_size, self.patch_size, 1],
            rates=[1, 1, 1, 1],
            padding="VALID"
        )

        patch_dims = patches.shape[-1]

        patches = tf.reshape(
            patches,
            [batch_size, -1, patch_dims]
        )

        return patches
```

`patch_dims = patches.shape[-1]`: Gets the number of pixel values per patch (768 for a 16×16 RGB patch).
`tf.reshape(patches, [batch_size, -1, patch_dims])`: Converts the 2D patch grid into a sequence, changing (1,14,14,768) to (1,196,768); -1 automatically calculates the number of patches.

Patch Encoder Layer

After extracting patches, each patch is projected into an embedding vector.

The patch encoder combines:

$$\text{Dense Projection} + \text{Position Embedding}$$

This gives the transformer both patch content and patch location.

Code

```python
class PatchEncoder(tf.keras.layers.Layer):
    def __init__(self, num_patches, projection_dim):
        super().__init__()

        self.num_patches = num_patches
        self.projection = tf.keras.layers.Dense(projection_dim)
        self.position_embedding = tf.keras.layers.Embedding(
            input_dim=num_patches,
            output_dim=projection_dim
        )

    def call(self, patches):
        positions = tf.range(start=0, limit=self.num_patches, delta=1)

        encoded = self.projection(patches) + self.position_embedding(positions)

        return encoded
```

Building the ViT Model

Now we combine the main components:

- input image,
- patch extraction,
- patch encoding,
- transformer encoder blocks,
- classification head.

Small ViT

This is a simplified Vision Transformer for experimentation.

Code

```python
def create_vit_classifier(
    input_shape=(32, 32, 3),
    patch_size=4,
    num_classes=10,
    projection_dim=64,
    transformer_layers=4,
    num_heads=4,
    mlp_dim=128):
    inputs = tf.keras.Input(shape=input_shape)

    patches = Patches(patch_size)(inputs)

    num_patches = (input_shape[0] // patch_size) * (input_shape[1] // patch_size)

    encoded_patches = PatchEncoder(
        num_patches=num_patches,
        projection_dim=projection_dim)(patches)

    x = encoded_patches

    for _ in range(transformer_layers):
        x = transformer_encoder(
            x,
            num_heads=num_heads,
            projection_dim=projection_dim,
            mlp_dim=mlp_dim)

    x = tf.keras.layers.LayerNormalization()(x)
    x = tf.keras.layers.GlobalAveragePooling1D()(x)
    x = tf.keras.layers.Dropout(0.3)(x)

    outputs = tf.keras.layers.Dense(num_classes, activation="softmax")(x)

    model = tf.keras.Model(inputs=inputs, outputs=outputs)

    return model
```

ViT Input and Patch Numbers

For CIFAR-10 images:

$$32 \times 32 \times 3$$

If we use 4×4 patches:

$$\frac{32}{4} \times \frac{32}{4} = 8 \times 8 = 64$$

patches.

Each patch contains:

$$4 \times 4 \times 3 = 48$$

numbers before projection.

Transformer Input

The image becomes a sequence of 64 patch tokens.

Global Average Pooling in Vision Transformers

- After the Transformer Encoders, the output shape is:

$$(64, 64)$$

  representing 64 patches, each containing 64 features.

- Our goal is to classify the entire image, not individual patches.

- **GlobalAveragePooling1D** averages each feature across all 64 patches:

$$z_j = \frac{1}{64} \sum_{i=1}^{64} x_{ij}$$

- This reduces the tensor from $(64, 64)$ to a single vector of 64 features:

$$(64, 64) \longrightarrow (64)$$

- The final **Dense(10, softmax)** layer uses these 64 features to predict probabilities for 10 image classes.

Code

```
vit_model = create_vit_classifier(
    input_shape=(32, 32, 3),
    patch_size=4,
    num_classes=10
)

vit_model.summary()
```

Code—Result

dense_9 (Dense)	(None, 64, 64)	8,256	dense_8[0][0]
add_5 (Add)	(None, 64, 64)	0	dense_9[0][0], add_4[0][0]
layer_normalizatio… (LayerNormalizatio…	(None, 64, 64)	128	add_5[0][0]
multi_head_attenti… (MultiHeadAttentio…	(None, 64, 64)	66,368	layer_normalizat… layer_normalizat…
add_6 (Add)	(None, 64, 64)	0	multi_head_atten… add_5[0][0]
layer_normalizatio… (LayerNormalizatio…	(None, 64, 64)	128	add_6[0][0]
dense_10 (Dense)	(None, 64, 128)	8,320	layer_normalizat…
dense_11 (Dense)	(None, 64, 64)	8,256	dense_10[0][0]
add_7 (Add)	(None, 64, 64)	0	dense_11[0][0], add_6[0][0]
layer_normalizatio… (LayerNormalizatio…	(None, 64, 64)	128	add_7[0][0]
multi_head_attenti… (MultiHeadAttentio…	(None, 64, 64)	66,368	layer_normalizat… layer_normalizat…
add_8 (Add)	(None, 64, 64)	0	multi_head_atten… add_7[0][0]
layer_normalizatio… (LayerNormalizatio…	(None, 64, 64)	128	add_8[0][0]
dense_12 (Dense)	(None, 64, 128)	8,320	layer_normalizat…
dense_13 (Dense)	(None, 64, 64)	8,256	dense_12[0][0]
add_9 (Add)	(None, 64, 64)	0	dense_13[0][0], add_8[0][0]
layer_normalizatio… (LayerNormalizatio…	(None, 64, 64)	128	add_9[0][0]
global_average_poo… (GlobalAveragePool…	(None, 64)	0	layer_normalizat…
dropout_7 (Dropout)	(None, 64)	0	global_average_p…
dense_14 (Dense)	(None, 10)	650	dropout_7[0][0]

Total params: 340,810 (1.30 MB)
Trainable params: 340,810 (1.30 MB)
Non-trainable params: 0 (0.00 B)

Compiling the ViT

A Vision Transformer can be trained using the same Keras workflow:

$$\texttt{compile()} \rightarrow \texttt{fit()} \rightarrow \texttt{evaluate()}$$

For CIFAR-10, we use:

- `AdamW` or `Adam`,
- sparse categorical crossentropy,
- accuracy.

Code

```
vit_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

Training the ViT

Training a Vision Transformer from scratch usually requires more data than training a small CNN.

For experimentation, we can still train a small ViT on CIFAR-10 to understand the workflow.

Important

Small ViTs trained from scratch may not outperform CNNs on small datasets. Their strength is often clearer with large datasets or pretrained models.

Dataset

```python
import tensorflow as tf
from tensorflow.keras.datasets import cifar10

# Load CIFAR-10
(x_train_full, y_train_full), (x_test, y_test) = cifar10.load_data()

# Normalize images to [0, 1]
x_train_full = x_train_full.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Flatten labels: (N, 1) -> (N,)
y_train_full = y_train_full.squeeze()
y_test = y_test.squeeze()

# CIFAR-10 has 50,000 training images
# Use 80% train, 20% validation
split = int(0.8 * len(x_train_full))

x_train = x_train_full[:split]
y_train = y_train_full[:split]

x_val = x_train_full[split:]
y_val = y_train_full[split:]

print("Train:", x_train.shape, y_train.shape)
print("Validation:", x_val.shape, y_val.shape)
print("Test:", x_test.shape, y_test.shape)
```

Code

```
history_vit = vit_model.fit(
    x_train,
    y_train,
    validation_split=0.2,
    epochs=10,
    batch_size=64
)
```

```
Epoch 1/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 413s 800ms/step - accuracy: 0.2636 - loss: 1.9648 - val_accuracy: 0.3569 - val_loss: 1.6938
Epoch 2/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 443s 802ms/step - accuracy: 0.3998 - loss: 1.6109 - val_accuracy: 0.4638 - val_loss: 1.4807
Epoch 3/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 445s 809ms/step - accuracy: 0.4678 - loss: 1.4651 - val_accuracy: 0.4478 - val_loss: 1.4875
Epoch 4/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 407s 814ms/step - accuracy: 0.4925 - loss: 1.3929 - val_accuracy: 0.4774 - val_loss: 1.4356
Epoch 5/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 397s 794ms/step - accuracy: 0.5161 - loss: 1.3386 - val_accuracy: 0.5014 - val_loss: 1.3607
Epoch 6/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 409s 819ms/step - accuracy: 0.5302 - loss: 1.2980 - val_accuracy: 0.5179 - val_loss: 1.3283
Epoch 7/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 396s 792ms/step - accuracy: 0.5445 - loss: 1.2566 - val_accuracy: 0.5231 - val_loss: 1.3039
Epoch 8/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 395s 790ms/step - accuracy: 0.5542 - loss: 1.2334 - val_accuracy: 0.5251 - val_loss: 1.3109
Epoch 9/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 450s 806ms/step - accuracy: 0.5669 - loss: 1.2027 - val_accuracy: 0.5479 - val_loss: 1.2579
Epoch 10/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 398s 797ms/step - accuracy: 0.5762 - loss: 1.1796 - val_accuracy: 0.5536 - val_loss: 1.2687
```

Evaluating the ViT

After training, evaluate the Vision Transformer on the test set.

This gives a fair comparison with the CNN trained on the same data.

Code

```
test_loss, test_acc = vit_model.evaluate(x_test, y_test)

print("ViT test loss:", test_loss)
print("ViT test accuracy:", test_acc)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 55s 169ms/step - accuracy: 0.5516 - loss: 1.2446
ViT test loss: 1.2445684671401978
ViT test accuracy: 0.5515999794006348
```

Current Subsection

1. Vision Transformers
   - Motivation
   - Image Patches
   - Patch Embeddings
   - Self-Attention
   - Transformer Encoder Block
   - Building a Small Vision Transformer
   - The Classification Token
   - CNNs vs. Vision Transformers
   - Pretrained Vision Transformers
   - Using KerasHub's pretrained ViT

What Is the [CLS] Token?

The original Vision Transformer adds a special learnable token to the patch sequence.

$$[\text{CLS}], \text{Patch}_1, \text{Patch}_2, \ldots, \text{Patch}_N$$

After the transformer blocks, the representation of the [CLS] token is used for classification.

Alternative

Some implementations instead use global average pooling across all patch tokens.

Code

```
batch_size = tf.shape(encoded_patches)[0]
projection_dim = encoded_patches.shape[-1]

cls_token = tf.Variable(
    tf.zeros((1, 1, projection_dim)),
    trainable=True
)

cls_tokens = tf.tile(cls_token, [batch_size, 1, 1])

tokens_with_cls = tf.concat(
    [cls_tokens, encoded_patches],
    axis=1
)

print("Before CLS:", encoded_patches.shape)
print("After CLS:", tokens_with_cls.shape)
```

```
Before CLS: (1, 196, 64)
After CLS: (1, 197, 64)
```

Note: One new token is added, and that token contains 64 learnable values.

Two Ways to Classify with ViT

After the transformer encoder, image information must be summarized.
Two common approaches are:

1. **Classification token**

$$\texttt{[CLS]} \rightarrow \text{Dense Classifier}$$

2. **Global average pooling**

$$\text{All Patch Tokens} \rightarrow \text{Average} \rightarrow \text{Dense Classifier}$$

For Experiment

Global average pooling is often simpler to implement, while `[CLS]` follows the original ViT design.

Current Subsection

1. Vision Transformers
   - Motivation
   - Image Patches
   - Patch Embeddings
   - Self-Attention
   - Transformer Encoder Block
   - Building a Small Vision Transformer
   - The Classification Token
   - CNNs vs. Vision Transformers
   - Pretrained Vision Transformers
   - Using KerasHub's pretrained ViT

CNNs vs. Vision Transformers

CNNs and Vision Transformers process images in fundamentally different ways.

Aspect	CNN	Vision Transformer
Basic unit	Pixel neighborhood	Image patch
Main operation	Convolution	Self-attention
Local bias	Strong	Weaker
Long-range relations	Indirect	Direct
Data efficiency	Often better	Often needs more data
Pretraining	Useful	Very important

Key Idea

CNNs are naturally local; Vision Transformers explicitly model relationships between patches across the image.

Inductive Bias

CNNs contain strong assumptions about images.

For example:

- nearby pixels are related,
- the same filter can be useful in different image locations,
- local features can be combined into larger structures.

These assumptions are called **inductive biases**.

Vision Transformers

ViTs use fewer image-specific assumptions and rely more heavily on data and learned attention.

Current Subsection

1. Vision Transformers
   - Motivation
   - Image Patches
   - Patch Embeddings
   - Self-Attention
   - Transformer Encoder Block
   - Building a Small Vision Transformer
   - The Classification Token
   - CNNs vs. Vision Transformers
   - Pretrained Vision Transformers
   - Using KerasHub's pretrained ViT

Training ViTs from Scratch vs. Pretraining

Vision Transformers often benefit strongly from pretraining.

Training from scratch may require:

- large datasets,
- significant computation,
- careful regularization,
- long training schedules.

Modern Practice

A common approach is to start from a pretrained transformer and fine-tune it on a smaller target dataset.

Transfer Learning with Vision Transformers

The transfer-learning idea is similar to pretrained CNNs:

$$\boxed{\text{Pretrained ViT} \rightarrow \text{Replace Classifier} \rightarrow \text{Train New Head} \rightarrow \text{Fine-Tune}}$$

Key Difference

The feature extractor now consists mainly of patch embeddings and transformer encoder blocks rather than convolution layers.

Where Do We Get Pretrained Vision Transformers?

Pretrained vision transformer models are available from several ecosystems.

- TensorFlow/Keras model libraries
- KerasHub
- Hugging Face Transformers
- TensorFlow Hub

Important

Always use the preprocessing and input resolution expected by the specific pretrained model.

Current Subsection

1. Vision Transformers
   - Motivation
   - Image Patches
   - Patch Embeddings
   - Self-Attention
   - Transformer Encoder Block
   - Building a Small Vision Transformer
   - The Classification Token
   - CNNs vs. Vision Transformers
   - Pretrained Vision Transformers
   - Using KerasHub's pretrained ViT

Pretrained Vision Transformer (ViT)

The pretrained ViT used here is an **ImageNet classifier**.

It was trained to recognize: 1000 ImageNet classes

These classes include many types of objects such as:

- animals and dog breeds,
- birds and insects,
- vehicles,
- fruits and foods,
- tools and household objects,
- clothing and other common objects.

For one input image, the model produces:

$$(1, 1000)$$

scores, one score for each possible ImageNet class.

Using KerasHub's pretrained ViT

```
!pip install -q keras-hub
```

```
import keras
import keras_hub
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
import urllib.request

preset = "vit_base_patch16_224_imagenet"

vit = keras_hub.models.ImageClassifier.from_preset(preset)

vit.summary()
```

```
Preprocessor: "vi_t_image_classifier_preprocessor"
```

Layer (type)	Config
vi_t_image_converter (ViTImageConverter)	Image size: (224, 224)

```
Model: "vi_t_image_classifier"
```

Layer (type)	Output Shape	Param #
images (InputLayer)	(None, 224, 224, 3)	0
vi_t_backbone (ViTBackbone)	(None, 197, 768)	85,798,656
get_item (GetItem)	(None, 768)	0
output_dropout (Dropout)	(None, 768)	0
predictions (Dense)	(None, 1000)	769,000

```
Total params: 86,567,656 (330.23 MB)
Trainable params: 86,567,656 (330.23 MB)
Non-trainable params: 0 (0.00 B)
```

Image Example

```
url = "https://github.com/pytorch/hub/raw/master/images/dog.jpg"
urllib.request.urlretrieve(url, "dog.jpg")

img = Image.open("dog.jpg").convert("RGB")
img = np.array(img)

plt.imshow(img)
plt.axis("off")
plt.show()
```



Preprocess the Image using the ViT's own preprocessing configuration

```
preprocessor = keras_hub.models.ViTImageClassifierPreprocessor.from_preset(preset)

img_processed = preprocessor(img)

print("Processed shape:", img_processed.shape)
```

```
Processed shape: (224, 224, 3)
```

Make the prediction

```python
import tensorflow as tf
import numpy as np
from tensorflow.keras.applications.imagenet_utils import decode_predictions

# Make prediction
predictions = vit.predict(np.expand_dims(img, axis=0))

# Convert raw scores to probabilities
probs = tf.nn.softmax(predictions[0]).numpy()

# Get predicted class
class_index = np.argmax(probs)

# Map ImageNet class index to class name
class_name = decode_predictions(
    np.expand_dims(probs, axis=0),
    top=1
)[0][0][1]

# Display results
print("Prediction shape:", predictions.shape)
print("Predicted class index:", class_index)
print("Predicted class name:", class_name)
print("Probability:", probs[class_index])
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
Prediction shape: (1, 1000)
Predicted class index: 258
Predicted class name: Samoyed
Probability: 0.5386765
```

Next Chapter

Next Chapter

So far, our models have mainly answered: *"What is in this image?"*
Next, we can ask: *"What objects are present, and where are they?"*

Object Detection

Congratulations!

Now you know about ViT!

